Lebanese International University

Department of Computer Science and Information Technology

PowerGrid LB

Electricity and Solar Management Platform for Homes in Lebanon

CSCI426 Advanced Web Programming
Project Phase 1 Report

Student	Ahmad Chhadi
Student ID	42230515
Campus	Nabatieh
Semester	Fall 2026
Repository	github.com/ahmad-chhadi/powergrid-lb
Live site	powergrid-lb.vercel.app

Abstract

Most Lebanese households depend on three sources of electricity: a few hours a day from the state grid (EDL), a neighbourhood generator subscription for the rest of the day, and increasingly solar panels. Information about these sources is scattered across WhatsApp groups, paper bills and word of mouth, so families cannot easily know when power will switch, which generator plan is fair, or whether solar would pay for itself.

PowerGrid LB is a front-end web application built with ReactJS and Tailwind CSS that brings these three sources into one place. It shows a 24-hour supply timeline for each area, lets users search, filter and compare generator networks and solar installers, estimates the monthly bill for any ampere plan, sizes a solar system from a list of household appliances, and presents consumption and bills in a dashboard with charts. The application has eight routes including a dynamic provider page (`/providers/:id`), is fully responsive from 360 px phones to desktop, and works in both English and Arabic with a right-to-left layout. The project is version-controlled with Git (13 commits) and deployed on Vercel. In this phase all data is sample data stored in JavaScript modules; a back end is planned for Phase 2.

1. Introduction

1.1 Problem

In many Lebanese areas the state grid supplies only a few hours a day. Generator subscriptions are priced by ampere level (5 A, 10 A, 15 A...) plus a price per kWh, and prices differ between providers. Households lack a single, clear view of today's supply hours, a fair way to compare providers, and a simple tool to judge whether solar panels are worth the investment.

1.2 Objectives

- Show today's power schedule (grid, generator, solar, outage) for the user's area, with the current hour and the next change.
- Let users find and compare providers by area, type, price, rating and supply hours.
- Give every provider a dedicated page with plans, an interactive bill estimate, reviews and a subscribe action.
- Help users size a solar system and see its cost and payback time.
- Track consumption by source and bill history in a dashboard.
- Work on any screen size and in Arabic as well as English.

1.3 Scope of Phase 1

Phase 1 is front end only, as required. Providers, schedules, usage and bills are realistic sample data in JavaScript files. The user's area, subscription and language choice are kept in the browser with `localStorage`. The contact form validates input and shows a confirmation without sending data to a server.

2. System Design

2.1 Architecture

The application is a single-page application (SPA). `main.jsx` wraps the app in three providers: **BrowserRouter** for navigation, **LanguageProvider** for the current language and text direction, and **PowerProvider** for the selected area and subscription. `App.jsx` renders the shared layout (Navbar, Footer) and maps each URL to a page. Pages are built from reusable components and read data through the `useData()` hook, which returns areas and providers in the current language.

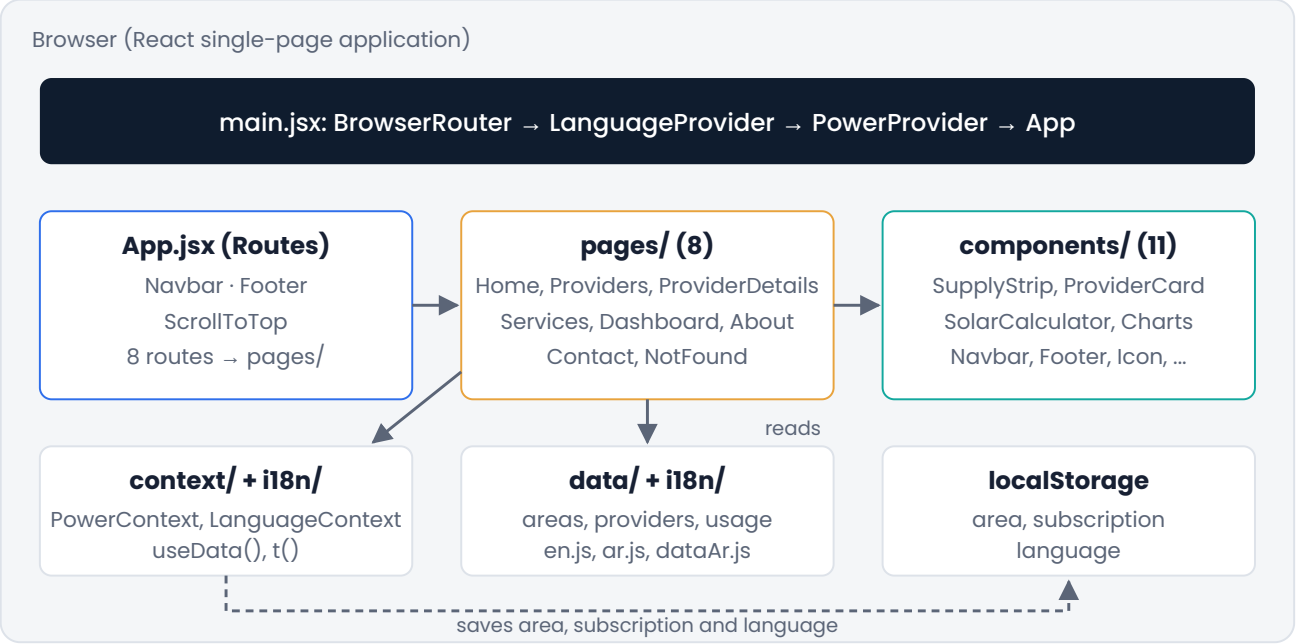


Figure 1. Application architecture and data flow.

2.2 Pages and routes

Route	Page	Purpose
`/`	Home	Today's 24-hour supply strip for the chosen area, features, how it works, local providers
`/about`	About	Problem statement, values, author and project timeline
`/services`	Services	Four services, interactive solar calculator, ampere guide table
`/providers`	Providers	Search, filter by area and type, sort by rating, price or hours (state kept in the URL)
`/providers/:id`	Provider details **dynamic**	Content chosen by the `:id` URL parameter: plans, bill estimator, coverage, reviews, subscribe
`/dashboard`	Dashboard	Subscription, key numbers, stacked bar chart, daily load chart, bills table
`/contact`	Contact	Validated contact form, contact details, FAQ

`*`	Not found	Friendly 404 page for unknown addresses

The required pages (Home, About, Services, Contact and a dynamic page) are all present; Providers, Dashboard and the 404 page are additions.

2.3 Main user flow

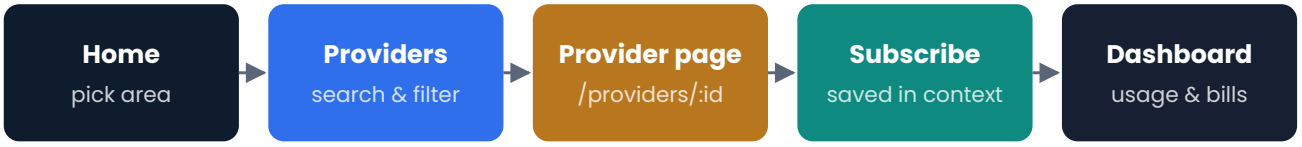


Figure 2. A typical user journey from choosing an area to tracking usage.

2.4 Folder structure

```
src/
  main.jsx            entry point: router and context providers
  App.jsx             layout and routes
  index.css           Tailwind import, design tokens, reusable classes
  components/  (11)   SupplyStrip, ProviderCard, SolarCalculator, Charts,
                      Navbar, Footer, PageHeader, Rating, Logo, Icon, ScrollToTop
  pages/        (8)   Home, About, Services, Providers, ProviderDetails,
                      Dashboard, Contact, NotFound
  context/            PowerContext: selected area and subscription
  i18n/               LanguageContext, en.js, ar.js, dataAr.js, useData
  data/               areas.js, providers.js, usage.js (sample data)
  utils/              schedule helpers, money format, page title hook
```

The project contains 34 source files and about 3,100 lines of code. Data, text and interface are kept separate: data in `data/`, all visible text in `i18n/en.js` and `i18n/ar.js`, and layout in components and pages.

2.5 Data model

Entity	Main fields	Notes
Area	id, name, schedule, note	`schedule` is a 24-character string, one character per hour: E grid, G generator, S solar, - outage
Provider	id, name, type, area, rating, since, hoursPerDay, phone, summary, features, reviews	6 generator networks and 3 solar installers
Generator plan	amps, fee, kwh	Monthly bill = fee + kWh used × price per kWh
Solar package	name, kw, battery, price	One-time price in USD

Subscription	providerId, type, amps or packageIndex, monthly, startedAt	Stored as numbers so it displays correctly in either language

2.6 State management

- **Local state (`useState`)** inside components: chosen plan, kWh slider, calculator appliances, form values and errors, mobile menu.
- **URL state (`useSearchParams`)** for provider filters, so a filtered list can be shared or bookmarked.
- **Shared state (React Context)**: `PowerContext` (area, subscription) and `LanguageContext` (language, direction, `t()` function), both saved to `localStorage` so they survive a page refresh.

2.7 UI/UX design

The visual identity is built from the subject itself: each power source has its own colour, used consistently in the supply strip, legends, charts and badges. The signature element is the **24-hour supply strip**, which shows the whole day at a glance and animates hour by hour when the page loads (the animation is disabled for users who prefer reduced motion). Headings use the **Archivo** typeface in a wider cut; Arabic text uses **IBM Plex Sans Arabic**.

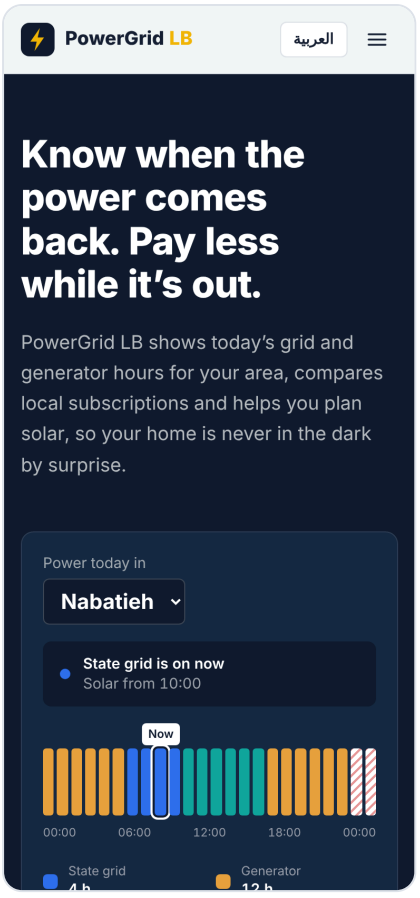
night	`#0f1c2e`	hero, footer, dark panels	**grid**	`#2f6feb`	state grid (EDL) hours
gen	`#e8a33d`	generator hours	**solar**	`#13a89e`	solar hours
cut	`#d64545`	outages, errors	**sun**	`#f2b705`	brand, main buttons
paper	`#f4f6f8`	page background	**ink**	`#172033`	text

Usability details include clear empty states with a way out (for example "No provider matches these filters" with a Clear button), inline form errors, visible keyboard focus, labels for screen readers, page titles for every route, and a mobile layout where the most important control (the plan picker) comes first.

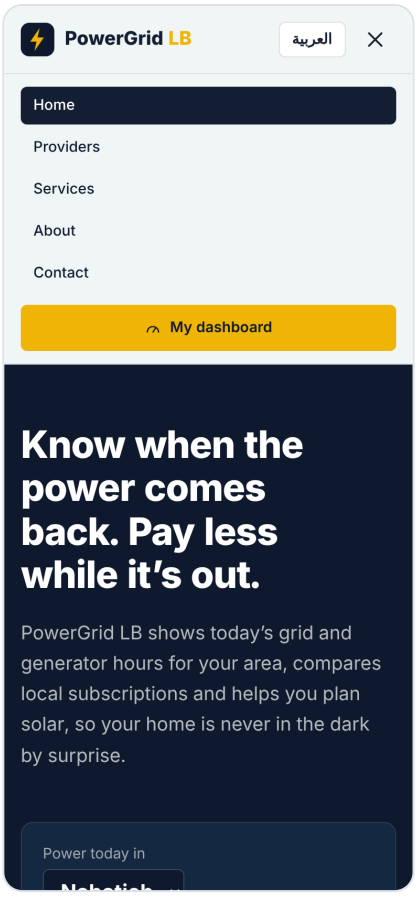
2.8 Responsive and bilingual design

Layouts use Tailwind's mobile-first breakpoints: one column on phones, two from `sm` (640 px), three or four from `md` / `lg`. The navigation collapses into a menu on small screens and wide tables scroll inside their own box. Every page was checked at 390 px and 1366 px widths with no horizontal overflow.

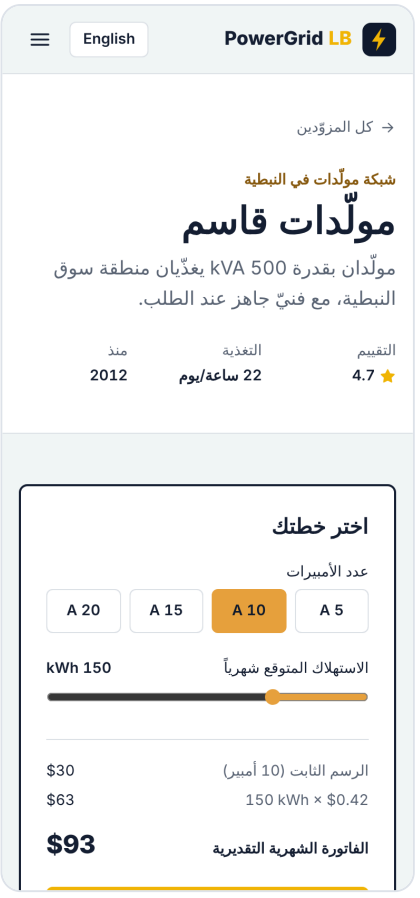
The language button switches the whole interface between English and Arabic. Setting `dir="rtl"` on the page mirrors the layout, and logical Tailwind classes (`ps-`, `pe-`, `start-`, `text-end`, `border-s`) make spacing follow the reading direction. Time-based visuals (the supply strip and charts) intentionally stay left-to-right like a clock.



Home on a phone (390 px)



Mobile navigation menu



Provider page in Arabic (RTL)

3. Technologies Used

Technology	Use in the project
ReactJS 19	Component-based user interface, hooks (useState, useEffect, useMemo, useCallback, useContext) and custom hooks
React Router 7	Client-side routing, dynamic route `/providers/:id` with `useParams`, URL filters with `useSearchParams`, active links with `NavLink`
Tailwind CSS 4	Utility-first styling, custom design tokens in `@theme`, reusable classes with `@utility`, responsive and RTL variants
Vite	Development server with instant reload and production build
SVG	Hand-written charts (stacked bars, load curve) and icon set, no chart or icon library
Context API + localStorage	Shared state for area, subscription and language that persists after refresh
Git and GitHub	Version control with 13 commits, one per feature step
Vercel	Hosting with automatic redeploy on every push; `vercel.json` rewrites all routes to `index.html` for the SPA
Google Fonts	Archivo and IBM Plex Sans Arabic

4. User Interface

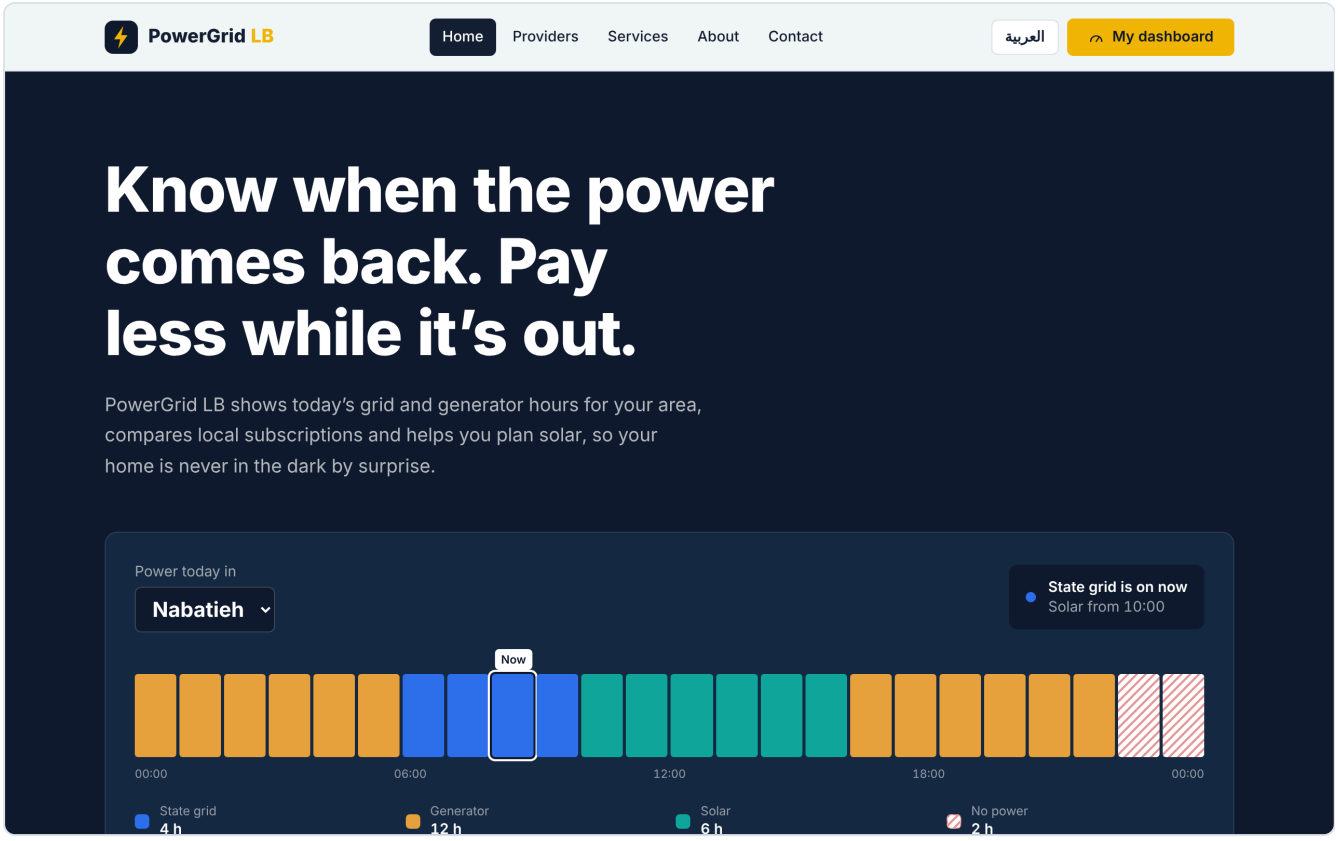


Figure 3. Home page: hero with the live 24-hour supply strip for the selected area.

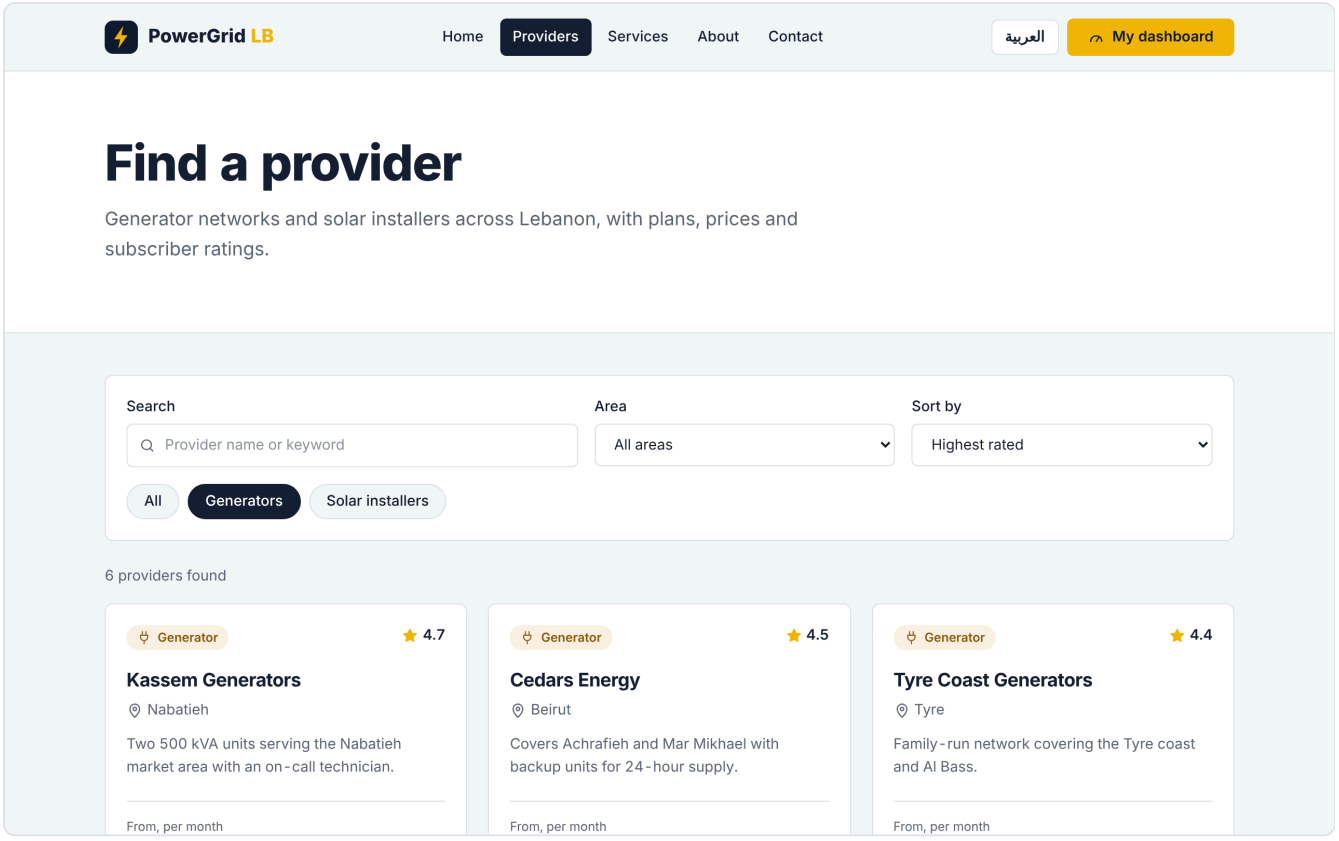


Figure 4. Providers page: search, area and sort filters, type chips and result cards.

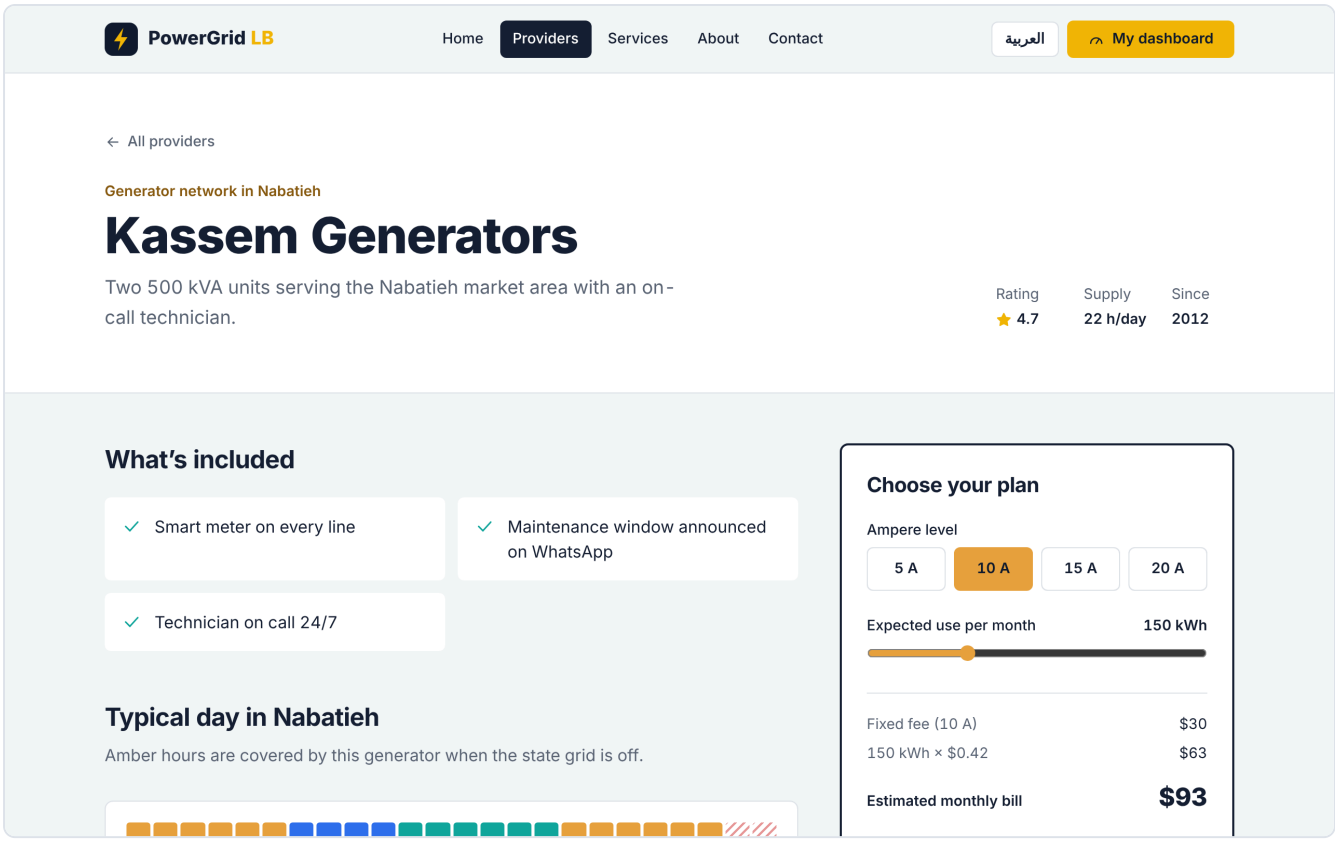


Figure 5. Dynamic provider page (/providers/kassem-generators) with the plan picker and bill estimate.

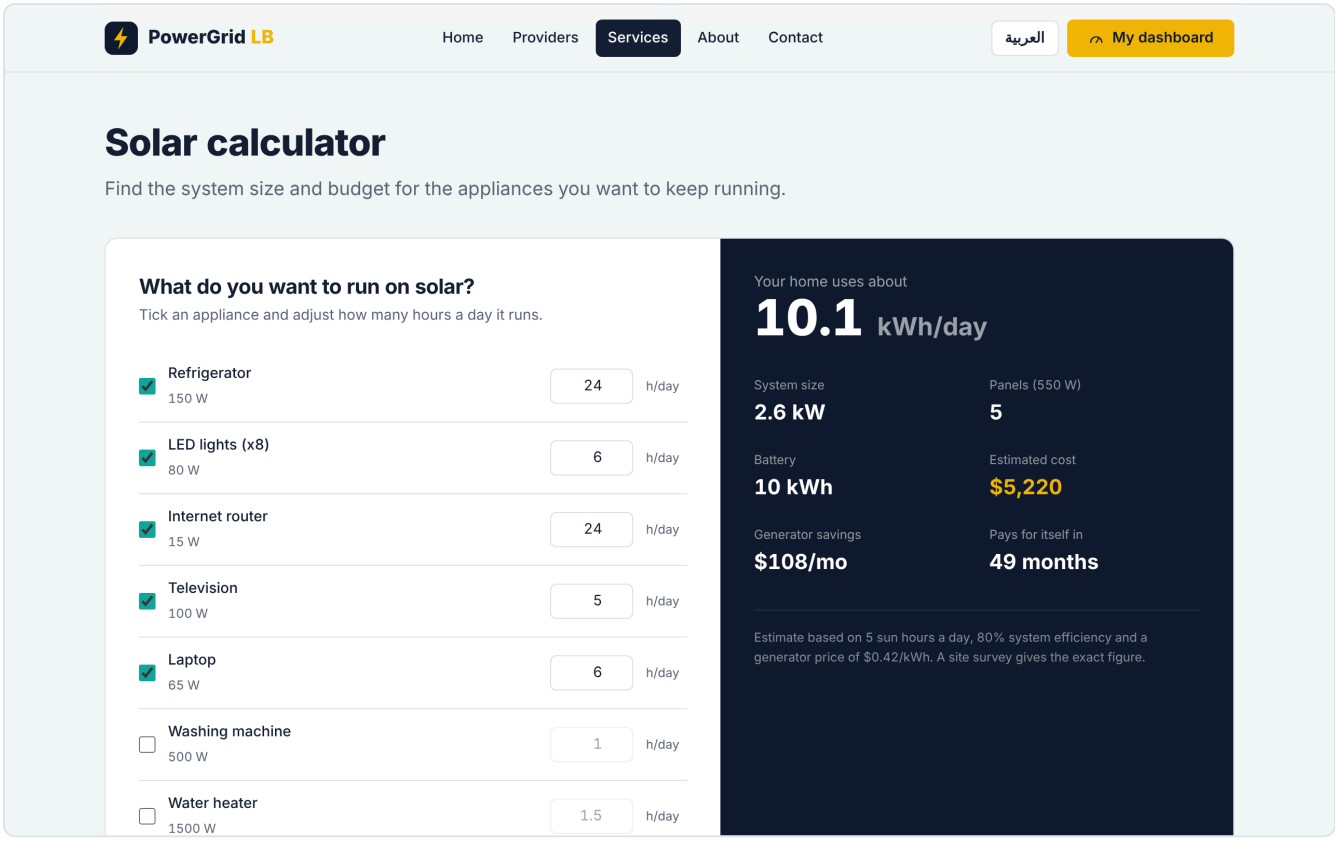


Figure 6. Solar calculator on the Services page.

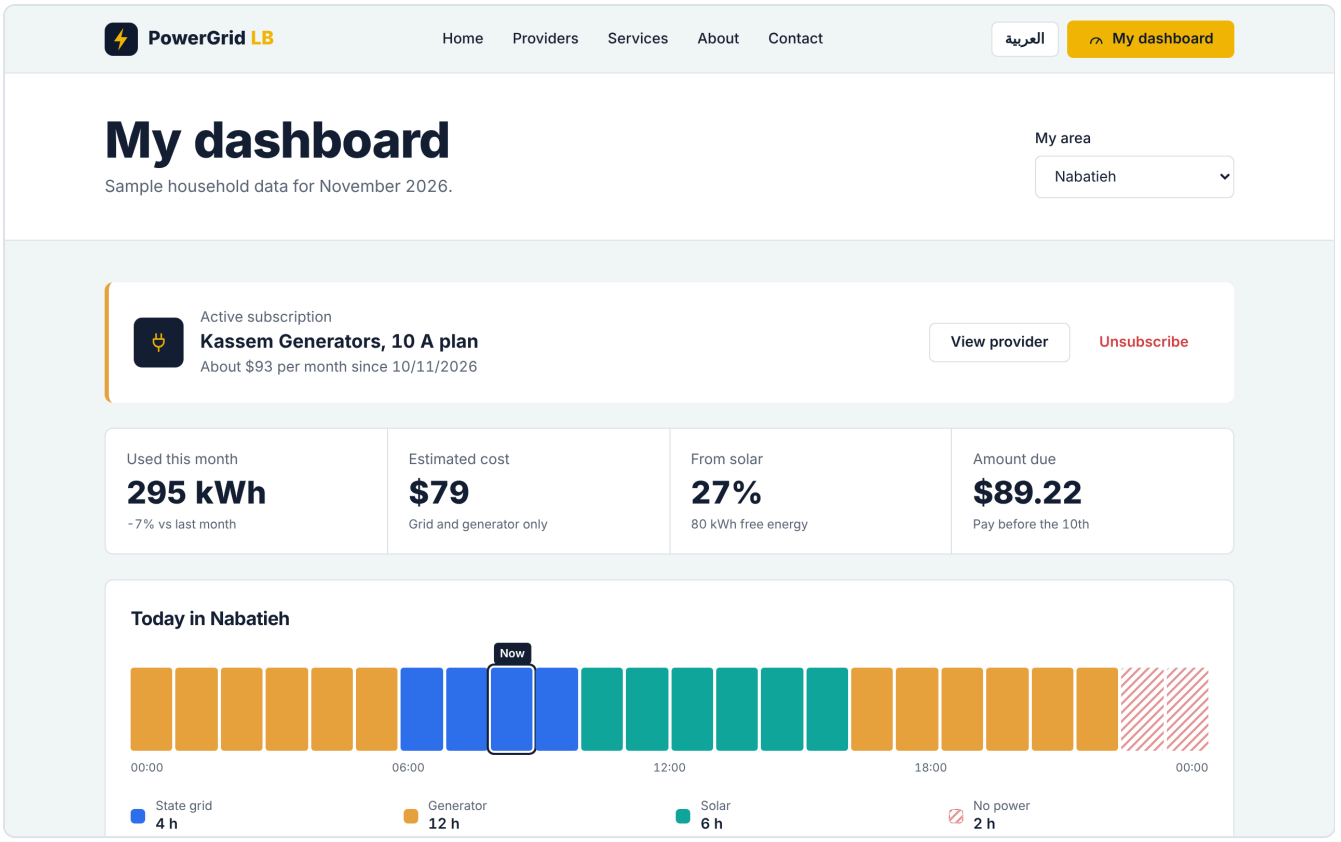


Figure 7. Dashboard: active subscription, key numbers and the supply for today.

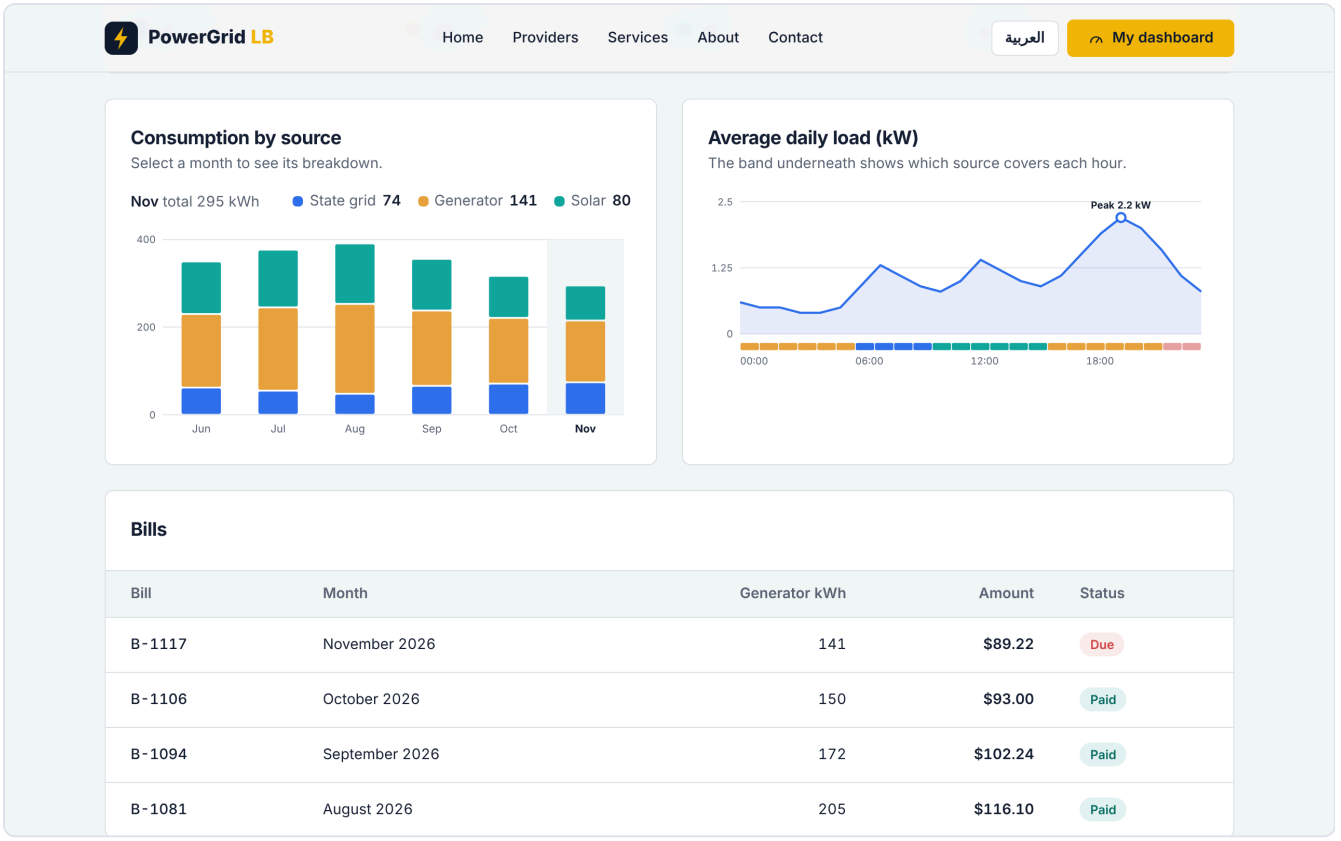


Figure 8. Dashboard charts (hand-written SVG) and bill history.

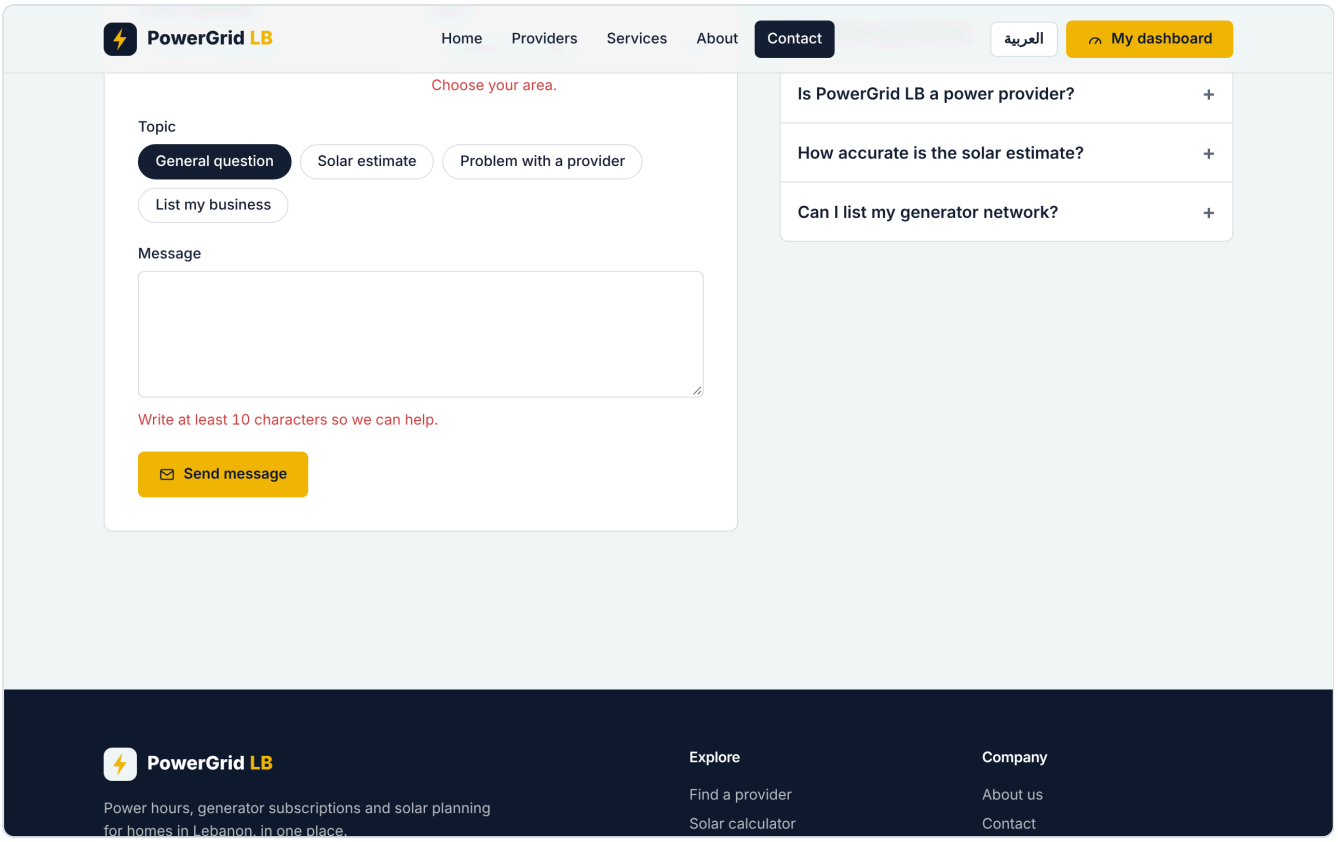


Figure 9. Contact form showing validation messages.

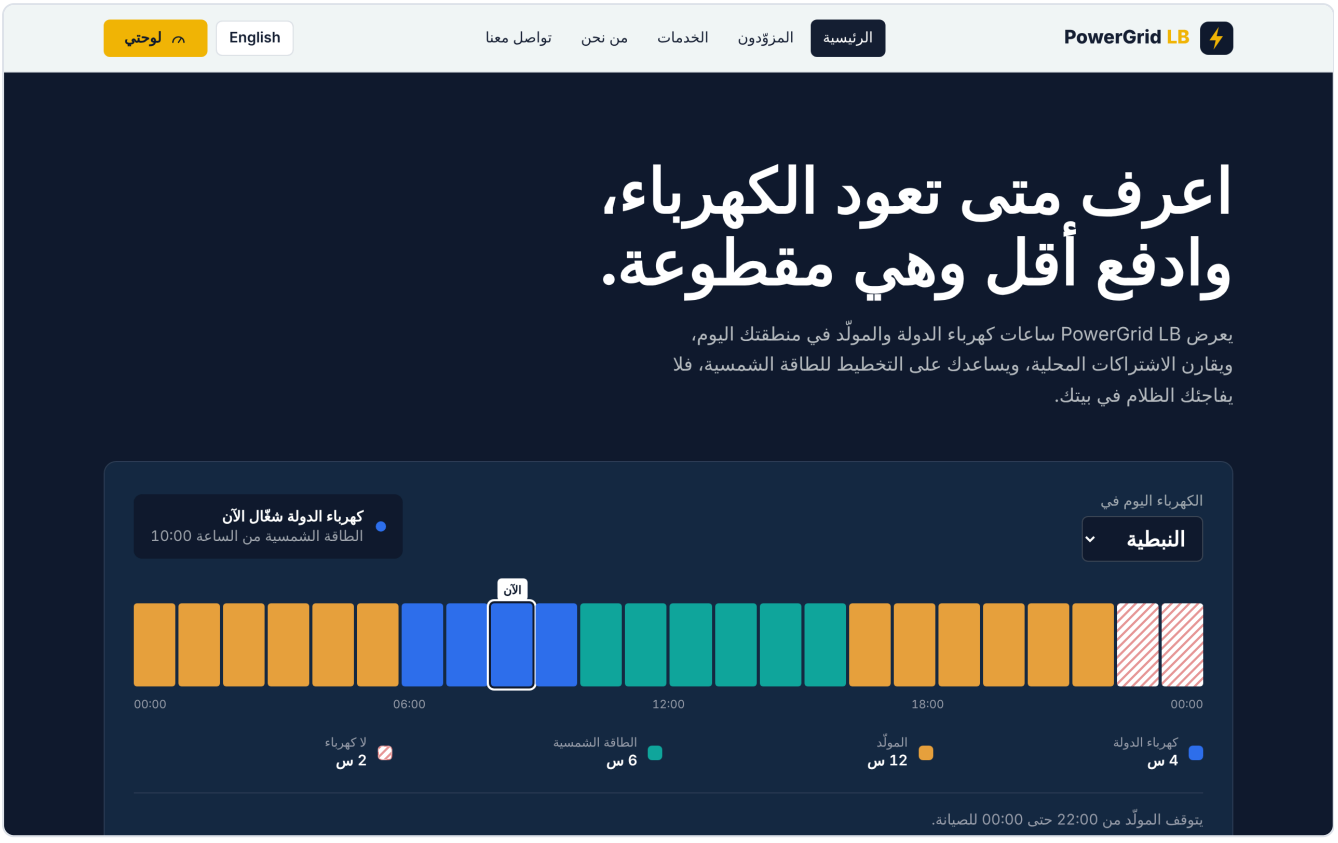


Figure 10. The Home page in Arabic with the right-to-left layout.

5. Key Code

5.1 Routes and the dynamic page

All pages are declared in one place. The `:id` segment makes the provider page dynamic: one component renders any of the nine providers, and an unknown id shows a "not listed" message instead of crashing.

src/App.jsx Route table

```
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/services" element={<Services />} />
          <Route path="/providers" element={<Providers />} />
          <Route path="/providers/:id" element={<ProviderDetails />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
```

src/pages/ProviderDetails.jsx Reading the URL parameter

```
  // the :id part of the URL decides which provider is shown
  const { id } = useParams()
  const provider = getProvider(id)
```

5.2 Search, filter and sort kept in the URL

src/pages/Providers.jsx Filtering with useSearchParams and useMemo

```
  // Filters live in the URL (?q=&area=&type=&sort=) so a filtered list can be shared or bookmarked.
  const [params, setParams] = useSearchParams()
  const q = params.get('q') ?? ''
  const area = params.get('area') ?? 'all'
  const type = params.get('type') ?? 'all'
  const sort = params.get('sort') ?? 'rating'

  const update = (key, value, fallback) => {
    const next = new URLSearchParams(params)
    if (!value || value === fallback) next.delete(key)
    else next.set(key, value)
    setParams(next, { replace: true })
  }

  const results = useMemo(() => {
    const text = q.trim().toLowerCase()
    return providers
      .filter((p) => area === 'all' || p.area === area)
      .filter((p) => type === 'all' || p.type === type)
      .filter((p) => !text || p.search.includes(text))
      .sort(sorts[sort] ?? sorts.rating)
  }, [providers, q, area, type, sort])
```

5.3 The 24-hour supply strip

The schedule string is split into 24 characters and each hour becomes a coloured cell. A small delay per cell creates the "switching on" animation.

`src/components/SupplyStrip.jsx` Rendering one cell per hour

```
          {schedule.split('').map((s, h) => (
            <div
              key={h}
              title={`${hourLabel(h)}: ${source(s).name}`}
              className={`flex-1 rounded-[3px] ${source(s).color} ${animate ? 'cell-on' : ''} ${
                h === now ? (dark ? 'ring-2 ring-white ring-offset-2 ring-offset-night' : 'ring-2
ring-ink ring-offset-2') : ''
              }`}
              style={animate ? { animationDelay: `${h * 35}ms` } : undefined}
            />
          ))}
```

5.4 Solar calculator

`src/components/SolarCalculator.jsx` Sizing formulas (constants are shown to the user under the result)

```
  const dailyKwh = items.filter((a) => a.on).reduce((sum, a) => sum + (a.watts * a.hours) / 1000, 0)
  const systemKw = Math.max(1, Math.ceil((dailyKwh / (SUN_HOURS * LOSSES)) * 10) / 10)
  const panels = Math.ceil((systemKw * 1000) / PANEL_WATTS)
  const batteryKwh = Math.max(5, Math.ceil((dailyKwh * NIGHT_SHARE) / LOSSES / 5) * 5)
  const cost = Math.round(systemKw * PRICE.perKw + batteryKwh * PRICE.perBatteryKwh + PRICE.install)
  const monthlySaving = dailyKwh * 30 * GENERATOR_KWH * 0.85
  const payback = monthlySaving > 0 ? Math.ceil(cost / monthlySaving) : 0
```

5.5 Form validation

Errors are stored as translation keys, so messages switch language together with the page.

`src/pages/Contact.jsx` Contact form validation

```
// returns an object with the translation key of the error for every invalid field
function validate(v) {
  const errors = {}
  if (v.name.trim().length < 2) errors.name = 'contact.errors.name'
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(v.email)) errors.email = 'contact.errors.email'
  if (v.phone && !/^\+?[\d\s-]{7,15}$/.test(v.phone)) errors.phone = 'contact.errors.phone'
  if (!v.area) errors.area = 'contact.errors.area'
  if (v.message.trim().length < 10) errors.message = 'contact.errors.message'
  return errors
```

5.6 Shared state with Context

src/context/PowerContext.jsx PowerContext: area and subscription saved to localStorage

```
export function PowerProvider({ children }) {
  const [state, setState] = useState(loadState)

  useEffect(() => {
    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(state))
    } catch {
      /* ignore */
    }
  }, [state])

  const value = {
    areaId: state.areaId,
    subscription: state.subscription,
    setArea: (areaId) => setState((s) => ({ ...s, areaId })),
    subscribe: (subscription) =>
      setState((s) => ({ ...s, subscription: { ...subscription, startedAt: new Date().toISOString() }
})),
    cancel: () => setState((s) => ({ ...s, subscription: null })),
```

5.7 Bilingual interface

The `t()` function returns text from `en.js` or `ar.js` and fills in values such as `{n}`. Changing the language updates `<html lang dir>`, which mirrors the whole layout.

src/i18n/LanguageContext.jsx LanguageContext

```
  // update <html lang dir> so the browser lays the page out right-to-left for Arabic
  useEffect(() => {
    document.documentElement.lang = lang
    document.documentElement.dir = dir
    try {
      localStorage.setItem(STORAGE_KEY, lang)
    } catch {
      /* ignore */
    }
  }, [lang, dir])

  // t('home.title') or t('providers.found', { n: 3 }) -> text in the current language
  const t = useCallback(
    (key, params = {}) => {
      const value = lookup(translations[lang], key) ?? lookup(translations.en, key) ?? key
      if (typeof value !== 'string') return value
      return value.replace(/\{(\w+)\}/g, (_, name) => params[name] ?? '')
    },
    [lang],
```

src/i18n/useData.js useData(): Arabic text merged over the English data

```
  const providers = baseProviders.map((p) => {
    const tr = providersAr[p.id] ?? {}
    // search works in both languages
    const search = [p.name, p.summary, tr.name, tr.summary].join(' ').toLowerCase()
    if (!ar) return { ...p, search }
    return {
      ...p,
      name: tr.name ?? p.name,
      summary: tr.summary ?? p.summary,
      features: tr.features ?? p.features,
      reviews: p.reviews.map((r, i) => ({ ...r, ...tr.reviews?.[i] })),
      packages: p.packages?.map((k, i) => ({ ...k, name: tr.packages?.[i] ?? k.name })),
      search,
    }
  })
```

6. Version Control and Deployment

The project was built step by step with one commit per feature, so the history shows how the application grew:

#	Commit message
1	Set up Vite, React Router and Tailwind CSS with design tokens
2	Add sample data, schedule helpers and shared PowerContext state
3	Add layout: responsive navbar, footer and icon set
4	Build home page with live 24-hour supply strip
5	Add providers page with search, filters and sorting
6	Add dynamic provider details page with plan picker
7	Add services page and solar calculator
8	Add dashboard with SVG charts and bill history
9	Add about page and contact form with validation
10	Add 404 page and Vercel SPA routing
11	Add Arabic and English language support with RTL layout
12	Add README with screenshots in both languages
13	Polish: page titles, remove unused code, project metadata

The repository is public at **github.com/ahmad-chhadi/powergrid-lb** and includes the full source code, assets and a README with description, setup instructions and screenshots. The site is deployed on Vercel at **powergrid-lb.vercel.app**; every push to the `main` branch redeploys it automatically.

Run locally: `npm install` then `npm run dev` and open `http://localhost:5173`. Production build: `npm run build`.

7. Testing

- Every route opened in English and Arabic at phone (390 px) and desktop (1366 px) widths, with no console errors and no horizontal scrolling.
- Search tested in both languages ("sun" and "شمس" return the solar installers), filters combined to reach the empty state, and the Clear button restores the list.
- Subscribe flow tested end to end: subscribe on a provider page, see it on the Dashboard, unsubscribe, refresh to confirm it is saved.
- Contact form submitted empty, with an invalid email and with valid data to check every message and the success state.
- Unknown addresses (`/nope` , `/providers/xyz`) show the 404 and "not listed" pages.

- Production build (`npm run build`) completes without errors.

8. Requirements Checklist

Requirement	How it is met
ReactJS with Bootstrap or Tailwind	React 19 with Tailwind CSS 4
At least 5 pages incl. a dynamic page	8 routes; dynamic `/providers/:id` plus search and filter page
Responsive design	Mobile-first layouts, mobile menu, tested at 390 px and desktop
Solves a real-world problem	Power schedules, provider comparison and solar planning for Lebanese homes
Git with commit history	13 commits, one per feature
Hosted on Vercel / Netlify / GitHub Pages	Vercel with SPA rewrite
README: description, setup, screenshots	Included, with English and Arabic screenshots
Original work, no unattributed template	Design, icons and charts made for this project; fonts credited

9. Conclusion and Future Work

PowerGrid LB turns a daily problem for Lebanese households into a clear, usable interface: users see when power is available, compare providers fairly, estimate bills, plan solar and follow their consumption, in English or Arabic, on any device. Phase 2 will add a back end with user accounts, a database of real providers and schedules, provider dashboards to publish outages, and real meter readings for the user dashboard.